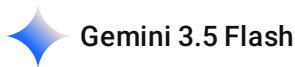




I am programming in R, I have a function called `abc(str)`. I pass a variable name ``string_one`` to the `abc()` function. tell me how to print out the name of the variable within the function. in this example, it should print out `string_one`



In R, when you pass a variable to a function, the function usually only sees the *value* of that variable. To capture the actual name of the variable passed by the user, you need to use a bit of R's "metaprogramming" toolkit.

You can achieve this by combining `substitute()` and `deparse()` .

The Solution

Here is how you can write the `abc()` function to print the variable's name:

```
r

abc <- function(str) {
  # Capture the expression and convert it to a character string
  var_name <- deparse(substitute(str))

  print(var_name)
}
```

How it works

- **`substitute(str)`** : Instead of evaluating the variable to get its value, `substitute` grabs the *expression* (the actual label or code) that the user typed into the function argument.
- **`deparse(...)`** : This takes that expression and turns it into a standard R character string so you can print it or manipulate it.

Example Usage

If you set up your variable and run the function, it will behave exactly as you want:

```
r
```

```
string_one <- "Hello World"
```

```
abc(string_one)  
# [1] "string_one"
```

💡 **A Quick Caveat:** If you pass a literal string directly into the function instead of a variable (like `abc("hello")`), `deparse(substitute(str))` will return `"\"hello\""` (including the quotes).

2026-06-10 16:20:40 You Asked 

is this the only way to achieve what I need?

💠 Gemini 3.5 Flash

While `deparse(substitute(str))` is the classic, textbook way to do this in R, it isn't the *only* way. Depending on which version of R you are using or if you are open to using external packages, there are a couple of modern alternatives that are often cleaner and safer.

Here are the two other main ways to achieve this:

1. The Modern R Way (R 4.0+)

If you are using a relatively recent version of R, a built-in function called `deparse1()` was introduced specifically to fix a sneaky bug in the older `deparse()` method.

If a variable name or expression is extremely long, the old `deparse()` will accidentally split it into a vector of multiple strings. `deparse1()` guarantees you always get a single string.

```
r  
  
abc <- function(str) {  
  var_name <- deparse1(substitute(str))  
  print(var_name)  
}
```

2. The tidyverse / rlang Way

If you already use the `tidyverse` or the `rlang` package in your projects, you can use `as_label(enquo())`. This is the standard approach used across modern data science packages like `ggplot2` and `dplyr`.

```
r
```

```
library(rlang)

abc <- function(str) {
  # enquo captures the variable, as_label turns it into a string
  var_name <- as_label(enquo(str))
  print(var_name)
}
```

Summary: Which should you use?

Method	Best Used For...	Why?
<code>deparse1(substitute(x))</code>	Standard base R scripts	Clean, built-in, and avoids the multi-line bugs of older R.
<code>as_label(enquo(x))</code>	Packages and Tidyverse code	Highly robust and handles complex code expressions beautifully.
<code>deparse(substitute(x))</code>	Legacy code	Works on ancient versions of R, but outdated now.

2026-06-10 16:22:11 You Asked 

thank you. tell me how to achieve exact the same in Python

Gemini 3.5 Flash

In Python, things work a bit differently. Python doesn't have a direct equivalent to R's `substitute()` because of how it handles variables. In Python, variables are just pointers (labels) stuck onto objects. When you pass a variable into a function, the function only receives the object itself, completely unaware of what label you used outside.

However, you can achieve the exact same result using a few different workarounds depending on your use case.

1. The Explicit Way (Recommended)

Because Python favors explicit code over "magic," the cleanest and most Pythonic approach is to pass the variable name as a string, or use a dictionary if you need to map names to values.

If you want the function to print the name, you simply pass it as a string:

```
python
```

```
def abc(var_name):  
    print(var_name)  
  
string_one = "Hello World"  
abc("string_one") # Prints: string_one
```

2. The Global Namespace Inspection (The "R-like" Way)

If you absolutely *must* mimic R's behavior where you pass the variable itself, you can inspect Python's global namespace (`globals()`) from inside the function to hunt down which variable name matches the value you passed in.

```
python
```

```
def abc(var_obj):  
    # Search the global variables for a name that matches the passed object  
    for name, value in globals().items():  
        if value is var_obj:  
            print(name)  
            return  
  
string_one = "Hello World"  
abc(string_one) # Prints: string_one
```

 **The Catch:** This method looks for the *object identity*. If two different variables happen to point to the exact same string or number in memory, this will just print the first one it finds.

3. Using an External Library (`varname`)

If you are doing heavy debugging or data science and want a robust, production-grade tool that behaves exactly like R, you can install a popular third-party library called `varname` .

First, install it via your terminal:

```
bash
```

```
pip install varname
```

Then use it in your code like this:

```
python
```

```
from varname import argname
```

```
def abc(str_var):  
    # Automatically retrieves the name of the argument passed to the function  
    var_name = argname('str_var')  
    print(var_name)  
  
string_one = "Hello World"  
abc(string_one) # Prints: string_one
```